INSPECTION & REPORTING ONLY · NO SOURCE MODIFIED

Gloobal — Full Engineering Audit

A complete review of the payment engine, the API, the database, the frontend, the security posture and the test suite, verified against live production where safe.

9 September 2026 | Commit `0d3989e` | Branch **main** | Tree **clean** | 910 tests run · **906 pass** | Overall progress **~65%**

§ Contents

STATUS KEY: WORKING | PARTIAL | PROTOTYPE | OPEN / RISKY | BROKEN | NOT VERIFIED | PRE-EXISTING | FIXED

01 Executive Summary

The one sentence that matters

Gloobal is a working cross-border payment prototype with a **genuinely well-built money path** and a **deliberately fake second factor**. The money is safe; the account is not.

The payment engine is better than most production systems of comparable age. The debit is an atomic conditional `$inc`, the whole transfer runs inside a Mongo transaction with a hand-written compensating path for deployments that lack one, idempotency is enforced by a partial unique index rather than a read-then-write, FX fails closed rather than guessing a rate, and both sides of every corridor are recorded in their own currency. Each of those is a correctness property systems routinely get wrong.

Against that, the platform **cannot hold real money today**, and the reason is not subtle. There is no SMS gateway. The one-time password is the constant `123456`. Anyone holding a target's Gloobal ID and mobile number can request a PIN-reset code, supply the constant, set a new PIN, and receive a valid seven-day bearer token. The PIN-reset route is also the one credential path that does not revoke the account's other sessions, so the original holder's devices stay signed in and silent. Neither path writes an audit record.

Beyond it, this audit found **one live functional bug not previously recorded** — a missing `await` that silently disables local-currency conversion on both coin-holder routes, confirmed against production — and a small set of currency-mixing defects in peripheral surfaces where different currencies are added as bare numbers. The core payment path does not have that defect; the code around it does.

90%	85%	80%	40%
Payments	Settlement	Coverage	QR security

25%	20%
Observability	Prod readiness

> **Verdict**
>
> **Production-ready as a demonstrable prototype. Not production-ready for real money**, and one finding is why. Eight tasks in the NOW column close the gap. Five are small. One needs a vendor.

---

02 Current Project Status

Repository baseline

Property	Value
Branch	**main**
HEAD	`0d3989e77aa867fcc03e89a32f0f8134de6e2b4e`
Working tree at audit start	CLEAN
Staged files	none
Untracked files	none
vs `origin/main`	0 ahead, 0 behind — **in sync**
Commits	105, spanning 11 Aug – 9 Sep 2026
Contributors	Aditya-Raj-oss (68) · gloobal-pay-gloobal (25) · Sanjeev santosh (11) · netlify[bot] (1)
Local / remote branches	18 / 10 — 17 merged-but-kept
Secrets tracked in git	NONE — only `secret.txt.example`

> **Parallel-session commits**
>
> Two commits on `main` (`ccb8e12` 8 Sep, `0d3989e` 9 Sep) have no local session transcript, so they were authored from another client or machine. Both are present and merged. Noted as a working-practice risk: concurrent sessions on `main` have already produced one recorded instance of two sessions holding different views of the tree.

Deployment — verified, not assumed

Target	Evidence	Status
Netlify frontend	Local `npm run build` of HEAD produced `index-B8AepYrZ.js`. Live site serves `/assets/index-B8AepYrZ.js`. Vite derives that hash from content, so the match identifies the deployed bundle as this exact commit.	CURRENT
Render API	`GET /api/coverage` answers 200 with `activeCountryRule: "has_users"`, a rule introduced in `2248d41` — the most recent commit touching `server/`.	CURRENT

Render only rebuilds when `server/` changes, so the two most recent (frontend-only) commits correctly left the API where it was. Cold start on the free tier is real: the first `/api/coverage` call took **23.2 seconds**. That is the documented spin-up, not a fault.

Live production readings (read-only, 09:10 UTC)

120	20	188	₹9.10M
Registered users	Countries with users	Successful payments	Global total spending

85/188	✓
Rows with sourceAmount	Coin supply reconciled

Every call was a `GET`. No account was created, no payment made, nothing written. The coin supply invariant `issued === heldByAccounts` holds exactly in production at 121,373.61 — a real, checked property.

03 Progress by Domain

Every figure is derived from what the code demonstrably does, not estimated.

Domain	Status	Evidence	Remaining major work
Authentication	PARTIAL	HMAC tokens, constant-time compare, 5-strike lockout, session revocation, production refuses to boot without a secret	Real SMS gateway; revoke on PIN reset; audit-log the login paths
Payments	WORKING	Atomic `$inc`; transaction with documented fallback; explicit `amountBasis`; client claims refused on mismatch; 188 live payments	Pagination; reconciliation for the best-effort tail
FX	PARTIAL	Live provider, Mongo-cached, 6h TTL, fails closed, transaction-time rate preferred historically	Bound staleness — `stale:true` is returned and discarded

Settlement	WORKING	Conditional `$inc`, ordered writes, typed refusals, revert reads its own amounts	Corridor seeding coverage; reconciliation report
Ledger	PARTIAL	Double-entry rows; balances read back from the writes that produced them	No uniqueness constraint; no replay; `metadata` is unvalidated
Receipts	PROTOTYPE	Rows written on every payment	**Nothing reads the collection.** Share receipts mislabel currency; best-effort loss is permanent
Creator Share	WORKING	Payee's own stored rate; split rounded per currency; both sides recorded; swapped party snapshot	Share-leg idempotency; receipt currency fix
Gloobal Coverage	WORKING	One server module; whole-collection aggregation; per-currency normalisation; exactness self-reported	Our Spending needs a disbursement record type that does not exist
Countries	WORKING	`accountCountryIso` resolver with dial-code fallback; bundled 194-country map; no invented currency	Run the backfill; one route still reads the raw field
Hooman Projects	WORKING	Full CRUD; ownership by document id; drafts 404; keyset paging; allow-listed uploads	Magic-byte validation; Mongo blobs will not scale
Security settings	WORKING	All three persist and change behaviour; PIN change verifies under lockout and revokes	App Lock is client-enforced only
QR	RISKY	Standards-compliant encoder; amount rejected not clamped; recipient resolved server-side	**No signature, nonce, expiry or server-side verification**
Coin / GEU	PARTIAL	Supply invariant reconciles live; all writes authenticated; the unbounded GEU growth loop is disabled behind a flag	Local-currency conversion broken; GLB-04 open
PayLater	PROTOTYPE	Limit and charge list built from real records	**No repayment path.** Limit and dues are cross-currency sums
Testing	PARTIAL	72 files; 910 tests run, 906 pass	Server suite unverified; ~180 source-shape assertions
Deployment	WORKING	Both targets confirmed current by independent evidence	Free-tier cold start; no staging
Observability	BROKEN	`AuditLog` wired with 15 call sites	Login, registration, PIN set and PIN reset write **nothing**. No metrics, tracing or alerting
Production readiness	NOT IMPLEMENTED	—	Real OTP; migrations; reconciliation; monitoring; DR

> **Overall ≈ 65%**
>
> The mean of the eighteen domain figures, weighted toward the money path because that is what the product is. Meaningful in one specific sense: **the things a payments platform must get right are mostly done, and the things that let it operate safely mostly are not.**

---

04 Frontend Architecture

Monolithic, screen-based, state-centralised, hybrid local-first. `frontend/` and `backend/` are not ES modules — `build_app.mjs` concatenates them in a declared order into one generated file sharing a single global scope. Top-level `var` names must be globally unique, React and icon imports use numbered aliases, and module order is semantically significant.

File	Lines	useState	useEffect
`screens/Dashboard/Dashboard.jsx`	4,162	122	15
`App.jsx`	4,000	84	22
`screens/SendMoney/SendMoney.jsx`	1,632	–	–
`components/dialogs/registerLogin.jsx`	1,026	–	–
`screens/Coverage/GloobalCoverageScreen.jsx`	920	–	–

Frontend source totals **21,909 lines**; the browser-side domain layer under `backend/` adds **7,991**. There is exactly one React context (`LedgerProvider`), wrapping the browser financial simulation. Everything else is `useState` in the two god components, distributed by props — `DashboardScreen` takes roughly forty.

APP STARTUP SEQUENCE

Launch splash	`launchSplash.jsx`
Session restore	`localStorage "gloobal.session.v1" · 30-day max age`
PIN / biometric stage	`a restored session NEVER lands on the dashboard`
Permissions / onboarding gate	`usePaymentLocation · LocationRequiredModal`

hydrateAccount() — one cycle	`GET /profile → reconcileBankBalance`
	`GET /assets → hydrateGrants`
	`GET /assets/paylater → reconcileDue`

balanceStatus: loading → ready | error	`three states, never two`
Dashboard	`4,162 lines · ~40 props`

Why balanceStatus has three states

The local ledger always holds *a* number, so a two-state read rendered "loading" and "confirmed" identically. A first login against a cold Render instance showed a confident, correctly formatted, entirely fictional balance. A figure is now shown only once the server has confirmed it.

Where state lives, and what is authoritative

Concern	Client location	Authority	Reconciliation
Session identity	`localStorage`	Client (not a credential)	30-day max age
Bearer token	same blob	**Server**	401 → clear + `gloobal:sessionExpired`
Balance	`FinancialCore` ledger	**Server** `User.balance`	`reconcileBankBalance` posts the difference as a real double-entry adjustment
Transactions / history	`App.jsx` state	**Server**	Re-fetched on poll tick
Country	`dialCountry`	**Server** `accountCountryIso`	Per-response
Coverage	screen state	**Server** `/api/coverage`	Refresh token
Security settings	`registeredUser`	**Server**	Replaced wholesale, never patched locally
Projects	screen state	**Server**	Per-fetch
Receipts	built from Transaction rows	**Server** party snapshot	—

QR "already used"	`usedQrCodes` Set	in-memory	**Client only** RISK	Lost on reload

> **The frontend's defining weakness — one structural duplication**
>
> A complete double-entry ledger simulation (`backend/domain/`, 7,991 lines: LedgerEngine, SettlementEngine, RiskEngine, TransactionOrchestrator, CreatorShareService, DisputeService, ProvenanceService) runs in the browser *alongside* the real server ledger. It is not dead code — the local balance is what the risk check reads. The two are kept in agreement by `reconcileBankBalance`, which posts the server's number in as an adjusting entry. Correct by construction, but it means every money-shaped rule exists twice and can only be kept honest by hand.

Secondary duplications, all currently in agreement: the 1,000-word summary limit (server and client, both commented as needing to change together); the QR dial-symbol alphabet (a private copy, because the concatenation build evaluates it before the shared constant is initialised — reaching for the shared value previously produced an empty `Set` and made every scan fail silently); and two 321-line country data files.

---

05 Backend Architecture

Two systems that share a repository

	`backend/`	`server/`
Runs in	the browser	Node on Render
Purpose	domain simulation	the real API
Storage	none	MongoDB Atlas
Build	concatenated into the bundle	deployed as-is
Size	7,991 lines · 80 files	12,670 lines · 42 source files

They must not be merged or renamed. `server/server.js` must stay at exactly that path or the Render deployment breaks.

REQUEST LIFECYCLE

express.json	64kb cap
CORS allowlist	no Origin passes · unknown Origin refused, not reflected
Security headers	CSP default-src 'none' · HSTS on production only · X-Powered-By removed
rateLimit(name, max, window)	process-local · first X-Forwarded-For hop only
requireAuth	HMAC verify → User lookup → credentialsInvalidatedAt
requireSelf(...sources)	compares by document id, never by symbolId string
Route handler	inline, own try/catch
withMongoTransaction(work)	transaction when available · compensating path when not

✗ no global error handler · ✗ no 404 handler · ✗ no health route

`GET / → HTML 404 · malformed JSON → HTML Bad Request`

The monolith

`server/server.js` is **8,274 lines** holding 62 routes, every model import, the auth primitives, the rate limiter, the audit recorder and all business logic. No router layer, no controller layer, no service layer. The send handler alone is ~1,100 lines. Domain logic that *has* been extracted lives in `server/lib/` (2,312 lines, ten modules) — and that extraction is the difference between the parts of this backend that are testable and the parts that are not.

Domain map

#	Domain	Source of truth	Separation	Known weakness
1	Authentication	Server	Mixed	**OTP is a constant**
2	Users / accounts	Server	Mixed	`countryIso` default masks real country until backfill
3	Payments	Server	Mixed — 1,100 lines in one handler	Best-effort tail unreconciled
4	FX	Provider → Mongo cache	CLEAN	Unbounded staleness
5	Settlement	Server	CLEAN	Corridor seeding coverage
6	Ledger	Server	Mixed	No uniqueness constraint
7	Receipts	Server	CLEAN	**Write-only**; share currency mislabel
8	Creator Share	Server (payee's rate)	Partly clean	Share leg not idempotent

9	Coverage	Server	CLEAN	Our Spending not computable
10	Countries / currency	Seeded table → bundled map	CLEAN	One route bypasses the resolver
11	Projects	Server	Mixed but cohesive	Bytes unvalidated
12	File attachments	Server (Mongo Buffer)	Mixed	2 MB in-document; will not scale
13	Security settings	Server	Mixed	App Lock client-enforced
14	Coin	Server	Mixed	Missing await · GLB-04
15	GEU	Server	Mixed	Correctly disabled behind a flag
16	PayLater	Server	Mixed	No repayment; cross-currency sums
17	Audit trail	Server	CLEAN	15 call sites; login not among them
18	Rate limiting	Process	Mixed	Volatile, per-instance

06 Database Architecture

Twenty-five Mongoose models. The financially significant ones:

Model	Authoritative	Reconstructable	Duplicates possible	Key constraints
User	YES — `balance` is the money	In principle from LedgerEntry; no replay exists	No	`symbolId`, `mobileNumber` unique
Transaction	YES	No	No	`referenceId` unique + **partial unique on (fromUserId, metadata.idempotencyKey)**
LedgerEntry	Derived record	Yes, from Transaction	YES — no constraint	Two non-unique compound indexes
Settlement	YES for pools	Yes, from pool history	No	`settlementId` unique
Receipt	No — **nothing reads it**	Yes	YES — no constraint	(transactionId, role) NOT unique
AssetSeed	Yes for interest	Partly	No	Partial unique on `transactionId`
Country	Yes when seeded	Yes, from the bundled map	No	`iso` unique
CountryCurrencyPool	YES	From Settlement	No	(countryIso, counterCurrency) unique
Project	Yes	No	Yes (by design)	Two compound indexes
ProjectAttachment	Yes	No	Deleted before replace	`projectId`, `ownerId`
AuditLog	Append-only	No	Yes (harmless)	Three indexes

Immutability is not enforced anywhere

`Transaction.metadata` is `Schema.Types.Mixed` — and it carries `sourceAmount`, `debitAmount`, `fxRate`, `cashback` and the party snapshot. **Financial metadata is unvalidated and mutable.** Mongoose applies no casting, no required fields and no type checks inside a Mixed field, so a bad write succeeds silently. Nothing writes it wrongly today; nothing prevents it.

HOW A PAYMENT TRAVELS THROUGH PERSISTENCE

Resolve both accounts · verify PIN under lockout	`5 strikes → 10 min, per account`
Resolve both currencies from each account's own country	`accountCountryIso → resolveCountry`
getRate(destination → source)	`fail closed on error → 502`
Compute BOTH sides, each rounded to its own precision `refuse if the client's claimed currencies disagree → 409`	
Idempotency pre-check + 15-second duplicate window	`index is the real guarantee`
— ATOMIC BLOCK —	`withMongoTransaction`
1. debit sender	`findOneAndUpdate({balance:{$gte:d}},{$inc:-d})`
2. Transaction.create(status 'success')	`before the credit, so settlement can attach`
3. settleCrossBorderPayment	`pool $inc conditional + Settlement.create`
4. credit receiver · 5. credit sender cashback	`balances read back from the writes`
6. LedgerEntry.create([debit, credit, cashback])	`— commit —`
7. AssetSeed.create — best effort	`outside the transaction · not retried`
8. mintShareLegAndReceipts — best effort	`outside the transaction · not idempotent · not reconciled`

> **Steps 1–6 are atomic. Everything after the commit is not, is not retried, and is not reconciled. That is GA-06.**

07 Financial Data Flow

Where the authoritative values live

Value	Stored as	Currency
What the receiver was credited (face value)	`Transaction.amount` / `.currency`	Receiver's
What the sender actually paid	`metadata.sourceAmount` / `.sourceCurrency`	Sender's

Same, legacy alias	`metadata.debitAmount` / `.senderCurrency`	Sender's
Rate applied	`metadata.fxRate`, `.fxRateSource`	—
Which side the person typed	`metadata.amountBasis`	—
Creator Share, receiver side	`metadata.cashback`	Receiver's
Creator Share, sender side	`metadata.cashbackCredit`	Sender's
Pool movement	`Settlement.*Amount`	Each pool's own

The classic failure modes, audited

Risk	Status	Finding
Source vs destination amount confusion	CLOSED	`amountBasis` names which side was typed; the other is computed, never accepted
Source vs destination currency confusion	CLOSED	Both derived from each account's own country; a client claim mismatch returns 409
Double conversion	CLOSED	One rate, one lookup; the inverse is derived rather than re-fetched
Missing conversion	CLOSED	Same-currency short-circuits to `fxRate: 1`
Client-side FX	CLOSED	Server-side only
Stale FX	OPEN	`getRate` returns `stale:true`; `server.js:5175` destructures only `{rate, source}`. Staleness unbounded
Incorrect historical FX	CLOSED	Coverage prefers each payment's own transaction-time rate
Duplicate transactions	CLOSED	Partial unique index; the losing racer's 11000 is caught and answered as a duplicate
Duplicate history rows	CLOSED	Share legs excluded from `/history`, included in `/transactions` with a type discriminator
Duplicate receipts	OPEN	No uniqueness constraint on `Receipt`
Balance mismatch	CLOSED	Balances read back from the writes that produced them
Ledger mismatch	PARTIAL	Rows correct; no constraint prevents duplicates, no replay verifies them
Settlement mismatch	CLOSED	Conditional `$inc`; revert reads its own amounts back
Creator Share interaction	CLOSED	Payee's own rate; both sides rounded to their own precision
Idempotency	CLOSED	At the database level
Retry behaviour	PARTIAL	Closed for the transaction; open for the best-effort tail
Timeout behaviour	CLOSED	`httpClient.js` distinguishes 401 from `status===0` — a cold start does not sign anyone out

Rounding

`toMinorUnit(value, currencyCode)` reads `Currency.decimals` through a synchronous cache. A JPY or KRW balance never receives a fraction it cannot hold. Each leg rounds in *its own* currency, and the sender's debit is the exact figure quoted rather than a re-rounding of the converted value.

08 Gloobal Coverage Architecture

`server/lib/coverageAggregation.js` (551 lines) is the single source. Nothing else may compute a Coverage number, and nothing else does.

Metric	Classification	Live value / note
Global Total Spending	SERVER-AUTHORITATIVE	₹9,095,563.13
Country Total Spending	SERVER-AUTHORITATIVE	`null`, never `0`, when unconvertible
Global Our Spending	UNAVAILABLE — HONESTLY	`available:false` with a written reason
Country Our Spending	UNAVAILABLE — HONESTLY	Same
Transactions / day	SERVER-AUTHORITATIVE	UTC calendar day, basis named in the response
Total users	SERVER-AUTHORITATIVE	120
Creator Share	SERVER-AUTHORITATIVE	7 buckets; percentages sum against the bucketed population
Active country	SERVER-AUTHORITATIVE	`activeCountryRule: "has_users"`, named in the response
Country lock / unlock	SERVER-AUTHORITATIVE	Derived from `active`
Live refresh	WORKING	Refresh token bumps the fetch
Currency display	SERVER-AUTHORITATIVE	`?currency=`, conversion server-side
Country search	CLIENT	Filters the returned list
Hooman Projects	SERVER-AUTHORITATIVE	Counts and list share one visibility rule

> **The best judgement call in this codebase**
>
> Our Spending is deliberately **not implemented as a number**. The module checked every mechanism that credits a user and established that each is funded by another user, not by the platform. It returns `null` plus the reason plus labelled diagnostics — never a stand-in. A plausible-looking number would have been easy and wrong.

Independent production verification. Per-country transaction counts sum to exactly **188**, matching `transactionsTotal`. Per-country user counts sum to exactly **120**, matching `/api/stats`. Both invariants hold.

`/api/coverage` and `/api/stats` are unauthenticated — a documented, reasoned choice, since the payload carries aggregates only, no ids, names or per-user amounts. A business-intelligence disclosure, not a vulnerability. `/api/stats` carries **no rate limit at all**, unlike `/api/coverage`.

---

09 Historical Data Integrity

188
Successful payments

85
Carrying sourceAmount (45.2%)

103
Not carrying it (54.8%)

THE FALLBACK CHAIN

metadata.sourceAmount
EXACT · 85 rows confirmed live

→

metadata.debitAmount
RECONSTRUCTABLE · same meaning, a field copy

→

amount
APPROXIMATE · receiver's face value on the sender's country

Class	Which rows	Recoverability
EXACT	The 85 with `metadata.sourceAmount`	Confirmed live
RECONSTRUCTABLE	Rows with `debitAmount` + `senderCurrency` but no `sourceAmount`	A field copy. No arithmetic, no rate lookup, no loss
APPROXIMATE	Rows with neither, on a genuinely cross-border payment	Falls back to the receiver's face value
UNRECOVERABLE	Rows with neither and no stored `fxRate`, cross-border	The transaction-time rate is gone. Today's rate would be a fabrication

> **The split between the last three is NOT VERIFIED**
>
> No route exposes a count of rows carrying `debitAmount`, and this audit performed no database reads. There is strong reason to expect it is small — for most of the pre-`debitAmount` era `Country` and `Currency` both held zero documents in production, every lookup missed, every account resolved to INR and `fxRate` was always 1, so those payments were domestic-equivalent and `amount` *is* the debit. That is an inference from the code's own recorded history, not a measurement.

Backfill: technically possible and safe for the reconstructable class. A field copy from `metadata.debitAmount` to `metadata.sourceAmount`, conditional on the target being absent and the source present. No arithmetic, so no new error. Conditions: a dry-run mode (the pattern `backfill-country-iso.mjs` already establishes), a backup first, and a hard refusal to touch any row where the two fields both exist and disagree. **No such script exists** — `server/scripts/` holds five scripts and none addresses `sourceAmount`.

Conflicting amounts: NOT VERIFIED. Detecting a row where `sourceAmount` and `debitAmount` disagree needs a database read.

`coverage.rowsWithRecordedSenderAmount` **understates** exactness, because the `exact` flag counts only `sourceAmount` while the pipeline also accepts the equally exact `debitAmount`. The reported 45.2% is a floor.

---

10 Security Audit

Live header verification — both targets

Header	API (Render)	Frontend (Netlify)
`Content-Security-Policy`	`default-src 'none'; frame-ancestors 'none'; base-uri 'none'; form-action 'none'`	`script-src 'self'` — **no unsafe-inline, no unsafe-eval**
`Strict-Transport-Security`	max-age 1y, includeSubDomains	+ preload
`X-Content-Type-Options`	nosniff	nosniff
`X-Frame-Options`	DENY	DENY

`Referrer-Policy`	no-referrer	strict-origin-when-cross-origin
`Permissions-Policy`	all denied	camera/geolocation/payment = self
`X-Powered-By`	removed	—

HSTS on the API proves `IS_PRODUCTION_DEPLOY` is true there, which proves `AUTH_TOKEN_SECRET` is set — the process would have called `process.exit(1)` otherwise.

Authorization probes (unauthenticated, read-only)

Endpoint	Result	Correct?
`GET /api/profile/TESTID`	401	YES
`GET /api/transactions/history/TESTID`	401	YES
`GET /api/users/available?symbolId=x`	400	YES — malformed ID refused before lookup
`GET /api/projects`	200	YES — published only, by query not post-filter
`GET /api/creator-share/distribution`	200	By design — counts only
`GET /api/coin/supply`	200	By design — aggregates only
`GET /api/geu/supply`	503	YES — prototype correctly disabled

Review by area

Area	Status	Evidence
Token signing	FIXED	HMAC-SHA256, one algorithm, no `alg` field to downgrade
Token comparison	FIXED	`timingSafeEqual` with a length check first
Token expiry	FIXED	7-day `exp`, checked every request
Token revocation	PARTIAL	Works on PIN **change**; **not on PIN reset**
Secret management	FIXED	Production refuses to boot without a ≥32-char secret
Session persistence	OPEN (accepted)	Bearer token in `localStorage` — XSS-reachable, mitigated by the strict frontend CSP
PIN handling	FIXED	bcrypt cost 10; 5 strikes → 10 min, per-account so IP rotation does not help
PIN reset	OPEN	Fixed OTP; no session revocation
Account enumeration	PARTIAL	Lookup routes hardened (GLB-17); `/api/otp/send` still distinguishes 409/404/400
Authorization	FIXED	`requireSelf` compares document ids — a rename cannot lock an owner out or hand data away
WebAuthn / passkeys	FIXED	Challenges expire and are single-use (GLB-25)
QR integrity	OPEN	See section 11
Client-trusted financial data	FIXED	Currencies, cashback rate and reference id all server-derived; a client claim is compared and refused
Rate limiting	PARTIAL	Real limiter with a bucket ceiling and eviction; process-local, reset by every cold start

Security headers	FIXED	Verified live, both targets
Sensitive logs	FIXED	No PIN, token or descriptor logged
Environment secrets	FIXED	Nothing tracked; `.env` gitignored and present only on disk
Upload validation	PARTIAL	Type allow-listed, size capped, filename sanitised; **bytes never checked**
Audit logging	OPEN	15 call sites; **login, registration, PIN set and PIN reset write nothing**

Status of prior findings

Finding	Subject	Status
GLB-04	Coin fiat leg labelled with hardcoded reserve currency	OPEN — deferred
GLB-05	Asset seeds of different currencies summed	OPEN — deferred
GLB-12	A released Gloobal ID is never reissued	FIXED
GLB-13	ID validated against its alphabet, not just its length	FIXED
GLB-14	Five wrong PINs cost ten minutes, not the account	FIXED
GLB-15	Concurrent settlements cannot overdraw a corridor	FIXED
GLB-16	Rate limiter documented as process-local and volatile	FIXED (as documentation)
GLB-17	Lookup routes disclose the minimum	FIXED
GLB-18	Transaction reference minted by the server	FIXED
GLB-19	Zero-decimal currencies never receive a fraction	FIXED
GLB-20	A registration OTP is single-use	FIXED
GLB-21	JSON-only security headers	FIXED — verified live
GLB-22	A pool cannot open under a meaningless currency pair	FIXED
GLB-24	Audit writes cannot abort the payment they describe	FIXED
GLB-25	WebAuthn challenge expires and is single-use	FIXED

Every GLB-12…GLB-25 fix has a named assertion in `server/tests/hardening-fixes.test.mjs`. **That suite was not executed** — it requires a live MongoDB cluster. The *fixes* are present in code (verified by reading); the *test evidence* is NOT VERIFIED.

---

11 QR Security Audit

THE PAYLOAD — 20 CHARACTERS, 8-SYMBOL ALPHABET

12 symbols
Gloobal ID · plaintext

+

7 symbols
amount, base-8 minor units · plaintext, editable

+

1 symbol
checksum · positional mod-8, algorithm ships in the bundle

Question	Answer
Contains a raw payment address?	YES — the Gloobal ID in clear
Payload signed?	NO — the checksum detects a misread; it authenticates nothing
QR data verified server-side?	NO — the server never sees the payload. The client decodes it and posts a `receiverSymbolId` like any other payment
Recipient identity resolved from the server?	YES — `GET /api/users/resolve` returns the payee's real name, masked mobile, own country and share rate
Amount / currency manipulable?	**Amount: yes** — plaintext, checksum recomputable. **Currency: no** — the QR carries none; the payee's registered country decides
Could a modified QR redirect funds?	YES — substituting another valid Gloobal ID sends the money there. Defences are non-cryptographic: the payer sees the resolved name/country/flag and must enter a PIN
Replay possible?	YES — `usedQrCodes` is an in-memory Set in `App.jsx:579`, lost on reload. Server-side the only brake is a 15-second same-amount window
Static vs dynamic treated differently?	NO — one format; zero amount = identity, non-zero = request
Expiration?	NONE
Nonce / replay protection?	NONE SERVER-SIDE

Current security level: integrity-checked plaintext

Not authenticated, not bound, not replayable-once. The exposure is bounded by three real things — the payer sees a server-resolved payee before confirming, a PIN is required, and the amount ceiling applies. It is **not** bounded by anything in the QR itself. The gap between this and "Gloobal-only QR scanning" as a security property is a signature, a nonce and a server-side verification step. None exists.

12 Hooman Projects Audit

Feature	Status	Evidence
Persistent model	FUNCTIONAL	`Project` + `ProjectAttachment` with real indexes
Ownership	FUNCTIONAL	Owner taken from the token, never the body; compared by document id
Create / Read / Update / Delete	FUNCTIONAL	All four routes, owner-scoped; delete is audit-logged
Category system	FUNCTIONAL	Eight frozen categories, validated against the allow-list
Search	FUNCTIONAL	Regex over title and summary, input escaped through `escapeRegExp`
Summary word limit	FUNCTIONAL	1,000 words server-side + a 24,000-character backstop

Upload	FUNCTIONAL	2 MB; 3 MB body cap for base64 inflation
Attachment persistence	FUNCTIONAL	Mongo Buffer; old bytes deleted before replace
Authorization (write)	FUNCTIONAL	`requireAuth` + owner match
Download authorization	FUNCTIONAL	Follows the project — a draft's file is as private as the draft
Draft / public behaviour	FUNCTIONAL	Excluded **in the query**, not post-filtered; 404 not 403 to strangers
File validation	PARTIAL	Type allow-listed (no SVG — correctly reasoned as a script container). **Bytes never inspected**
Production suitability	PARTIAL	Correct and safe, but 2 MB blobs inside documents is not a file-storage strategy

Two details worth naming

`.lean()` returns a BSON `Binary` rather than a Node `Buffer`, which previously made every download serve `{"buffer": {…}}` as JSON — the fix normalises it and sets `Content-Length` from the buffer actually written, not the stored `byteSize`. And `sanitiseFilename` strips path separators, control characters and both quote marks before the name reaches a `Content-Disposition` header.

13 Security Screen Audit

Setting	Persists	Affects behaviour	Survives reload	Survives re-login	Other sessions	Secure backend path
Biometric Login	YES	YES — routes `requireBiometric` to the PIN; never removes a check	Yes	Yes	Yes	`PATCH /api/profile/security/:symbolId` · `requireAuth` + `requireSelf` · boolean-typed
App Lock	YES	YES — a `visibilitychange` listener returns a backgrounded session to the PIN stage after a grace period	Yes	Yes	Yes	Same route
Change PIN	YES	YES	Yes	Yes	REVOKES THEM	`POST /api/pin/change` · verifies the current PIN under the same 5-strike budget, stamps `credentialsInvalidatedAt`, mints a replacement token

Two honest limits

App Lock is client-enforced — the listener lives in the page and a local attacker with devtools bypasses it. Inherent to a browser app, and the setting is still worth having. And the "ask for your PIN every time the app opens" behaviour the old label described *already happens unconditionally* — a restored session always lands on the PIN stage. The switch was deliberately given the gap that genuinely existed (re-locking after backgrounding) rather than being wired to make an existing credential check optional.

---

14 Testing Coverage

Suite	Files	Tests	Executed here	Result
`tests/` — frontend + domain	37	695	YES	693 pass · **2 fail**
`financial-principles-tests/`	14	215	YES	213 pass · **2 fail**
`server/tests/`	21	–	NO	Requires a live MongoDB cluster
Total	**72**	**910 run**		**906 pass · 4 fail**

Why the server suite was deliberately not run

Each file creates and drops a throwaway database *on the production Atlas cluster*. That is a write to production infrastructure, which this audit's terms exclude. Its coverage is reported from source inspection and classified NOT VERIFIED.

The four failures — all stale tests, no application defect

Test	Assertion	Cause
`receipt-counterparty.test.mjs:375`	badge must be a landscape 46×40 chip; got 26×26	Contradicts `4fbc7dd`, which changed the receipt flag to a disc **on request**
`receipt-counterparty.test.mjs:548`	chip must be declared landscape	Same commit
`coinLedger.test.mjs:223`	coin history direction	GC→GEU rename (`033d7f3`)
`coinLedger.test.mjs:240`	expected `'GC'`, got `'GEU'`	Same rename

All four encode a **superseded requirement**. The application behaves as most recently specified. Real maintenance debt and a red CI, but not bugs. **They were not fixed, per the audit's terms.**

Coverage by critical area

Area	Rating	Basis
Cross-border payments	GOOD	Five dedicated server suites
Duplicate / idempotency	GOOD	Two suites plus the database constraint itself
Historical aggregation	PARTIAL	438-line suite exists, not run
Coverage	PARTIAL	Same
Country resolution	GOOD	Two suites plus a live cross-check in this audit

Projects	PARTIAL	354-line suite, not run
Security settings	PARTIAL	Inside `security-controls`, not run
QR payments	PARTIAL	Encoding well covered. **Security properties untested — there are none to test**
Receipts	PARTIAL	Rendering covered; **the Receipt collection has no test at all**
Creator Share	GOOD	Four root suites plus `merchant-share-flow`
Concurrency	GOOD ON PAPER	100-concurrent-send suite exists, not run
Observability	NO MEANINGFUL COVERAGE	Tests the recorder, not what calls it
Rate limiting	NO MEANINGFUL COVERAGE	—

Tests that assert existence rather than behaviour

Roughly **180 of ~1,000 root assertions** match source text with a regex rather than exercising the code. The heaviest are `receipt-counterparty` (17), `location-gate` (13), `money-path` (12), `qa-2026-08-24` (11), `geu-conversion` (10). They break on any refactor that preserves behaviour and pass on any refactor that breaks it — both failing receipt tests are of exactly this kind. Not worthless (several pin invariants that would otherwise be untestable in a concatenated build), but they must not be counted as behavioural coverage.

15 Production Readiness

#	Category	Rating	Evidence
A	Financial correctness	GREEN	Atomic debit, dual-currency contract, per-currency rounding, fail-closed FX. 188 production payments; coverage invariants hold
B	Data integrity	ORANGE	Transaction and settlement well constrained. **No uniqueness on LedgerEntry or Receipt; metadata unvalidated; cross-currency sums in three surfaces**
C	Security	RED	Excellent headers, tokens, authorization and rate limiting — and `123456` **as the second factor with no SMS gateway**
D	Authentication	ORANGE	Token mechanism sound; the credential behind it is not
E	Authorization	GREEN	`requireSelf` by document id everywhere; probes returned 401/403 correctly; drafts 404 to strangers
F	Reliability	ORANGE	Payment path robust. **No health route, no global error handler, no 404 handler**; cold start 20–50s
G	Concurrency	GREEN	Conditional `$inc` on balances and pools; partial unique index; 100-concurrent-send test exists
H	Observability	RED	15 audit call sites and **none on login, registration, PIN set or PIN reset**. No metrics, tracing or alerting
I	Persistence	YELLOW	Atlas with transactions; sound indexes. No backup policy in the repo, no migration framework
J	File storage	ORANGE	Correct and safe, but 2 MB blobs inside Mongo documents. **Bytes never validated**
K	Frontend reliability	ORANGE	Error boundaries, three-state balance, 401-vs-timeout. Against that: **932 KB single chunk**, 4,000- and 4,162-line components
L	Backend architecture	ORANGE	`lib/` extractions genuinely clean. `server.js` **is 8,274 lines with no layering**
M	Test coverage	YELLOW	910 run, 906 pass. **Server suite unverified; 4 stale failures; ~180 source-shape assertions**
N	Deployment	GREEN	Both targets confirmed current by independent evidence
O	Disaster / recovery	RED	**No documented backup, restore procedure, rollback runbook or migration framework.** Five ad-hoc scripts

3	2	7	3
Green	Yellow	Orange	Red

16 Bugs and Complex Errors

> **New finding — confirmed against live production**
>
> `lib/countryCurrency.js:96` declares `localCurrencyFor` as `async`. `server.js:6588` and `:6680` call it **without** `await`. The result is a Promise, which (1) serializes to `{}`, (2) never equals `reserveCurrency` so the code always takes the FX branch, and (3) is passed to `getRate` as a currency code, which fails and is caught — leaving `localHeld` as `null`.
>
> Live proof: `{"countryIso":"IN","holders":6,"held":87876.3,"localCurrency":{},"localHeld":null}` — every country row, always. **The feature has never worked, and it fails into its own error branch so nothing surfaces.**

DATA ERRORS

- **Cross-currency sums, three places.** `server.js:4336` adds seed interest across currencies then credits the raw total (prior finding **GLB-05**, still open — this creates or destroys value). `server.js:6157` sums `totalSent` / `totalReceived` across currencies (latent — `App.jsx:1872` reads only `transactions`). `server.js:4393` sums PayLater assets across currencies, with charges in the receiver's currency and credits in the sender's.
- **Share-leg receipts pair the wrong currency.** `merchantShareFlow.js:250-256` passes `amount: cashback` (payer's currency) with `currency` (destination currency). The share `Transaction` twelve lines above correctly uses `cashbackCurrency || currency`.
- **Two country-resolution rules in one server.** `/api/coin/holders` groups on the raw `$countryIso` field — the exact defect `coverageAggregation.js` exists to fix.
- **Coverage exactness understated.** The `exact` flag counts only `sourceAmount`, while the pipeline also accepts the equally exact `debitAmount`. 85/188 is a floor.

LOGIC ERRORS

- **Duplicate object key** — `server.js:395` and `:398` both set `replacedBy`. Same value, harmless, and a marker that the literal was edited without being read whole.
- `Receipt` **is write-only.** No route queries it.
- **PayLater has no repayment path** — the route says so: "Nothing repays a charge yet."

CONCURRENCY

The money path is sound. Two gaps outside it: `mintShareLegAndReceipts` runs *after* the atomic transaction, is not idempotent and has no uniqueness constraint, so a crash between commit and that step leaves a successful payment with no receipt trail, permanently. And `/api/assets/claim-interest` marks seeds claimed one at a time then credits the balance *outside any transaction* — a crash between the two marks interest paid without paying it.

FRONTEND

- 932 KB single JavaScript chunk, no code splitting; Vite warns on every build.
- `Dashboard.jsx` takes ~40 props; 122 `useState` in one component.
- `usedQrCodes` is per-session in-memory state doing a security job.

BACKEND & DATABASE

- No global error handler, no 404 handler, no health route — malformed JSON returns HTML, breaking the JSON contract every other response keeps.
- `Transaction.metadata` is unvalidated `Mixed` and carries financial fields.
- Rate limiter process-local and volatile; on a free tier that sleeps, a cold start is a fresh budget. `/api/stats` carries no rate limit at all.
- No uniqueness constraint on `LedgerEntry` or `Receipt`. No migration framework.

FILES AND UPLOADS

Bytes never validated against the claimed content type — mitigated by the allow-list, `Content-Disposition: attachment`, `nosniff` and `default-src 'none'`, but the file stored is not necessarily the type recorded. 2 MB blobs sit inside Mongo documents.

---

17 Architectural Strengths

Each claim is supported by code or a production reading.

#	Strength	Evidence
1	**Server-authoritative financial values**	Currencies from each account's own country, cashback rate from the payee's record, reference id from the server. A client's claimed currencies are compared and a mismatch returns 409
2	**Genuine transaction atomicity**	Debit, transaction row, settlement, credit, cashback and ledger lines commit together — with a hand-written compensating path tracking exactly which steps applied
3	**Idempotency enforced by the database**	A partial unique index, not a read-then-write. The losing racer's duplicate-key error is caught and answered with the winner's row
4	**Concurrency handled where it matters**	Balances and pools move by conditional `$inc`. Two payments racing out of one account cannot both pass
5	**The account-country resolver**	One rule shared by the send route, the resolve route, the settlement engine and the coverage aggregation — the country the API reports and the one it settles in cannot diverge
6	**Fail-closed FX**	`getRate` throws rather than returning 1.0; the send route turns that into a 502 rather than moving the wrong amount
7	**Per-currency precision**	`Currency.decimals` read for real. A JPY balance never carries a fraction
8	**Coverage refuses to fabricate**	Our Spending returns `null` plus a written reason plus labelled diagnostics
9	**Coherent project visibility**	Drafts excluded in the query, 404 rather than 403 to strangers, ownership by document id
10	**A disciplined API client boundary**	`httpClient.js` distinguishes 401 from `status===0` — the reason a cold start does not sign people out
11	**The GEU growth loop is disabled rather than shipped**	An unresolved authorization question turned into a 503 behind an environment flag instead of a live endpoint
12	**Security headers, verified live on both targets**	The frontend CSP has neither `unsafe-inline` nor `unsafe-eval`
13	**Comments that record the failure, not the feature**	Large parts of this codebase explain the bug a line prevents. It is why this audit could reconstruct intent so precisely, and it is a real asset

---

18 Architectural Weaknesses

#	Weakness	Impact	Evidence
1	**Prototype authentication in a production deployment**	CRITICAL	Fixed OTP, no SMS gateway. The whole recovery path is open
2	**Monolithic server**	HIGH	`server.js` 8,274 lines, 62 routes, no layering. The send handler alone is ~1,100 lines
3	**Duplicated financial logic across the client/server boundary**	HIGH	A 7,991-line browser ledger mirrors the server's, reconciled by hand
4	**Observability near zero**	HIGH	Login and PIN reset write no audit record. No metrics, tracing or alerting

5	**Insufficient database constraints**	HIGH	No uniqueness on LedgerEntry or Receipt; financial metadata unvalidated Mixed
6	**Oversized frontend components**	MED-HIGH	`Dashboard.jsx` 4,162 / 122 `useState`; `App.jsx` 4,000 / 84
7	**Best-effort tail with no reconciliation**	MED-HIGH	Seeds and receipts can be silently lost after a successful payment
8	**No migration framework**	MED-HIGH	Five ad-hoc scripts; schema changes rely on read-time derivation forever
9	**Incomplete event modelling**	MEDIUM	`TransactionEventOutbox` exists only in the browser layer; the server has no outbox
10	**Hardcoded business rules**	MEDIUM	Default balance 10,000; cap 5,000,000; 15-second window; 1,000-word summary; 7 buckets — all literals
11	**Weak domain boundaries**	MEDIUM	Coin, GEU, PayLater and assets interleaved in one file
12	**Concatenation build**	MEDIUM	Global scope, numbered aliases, order-sensitive. Already caused one silent total failure (`new Set(undefined)` in the QR decoder)
13	**No code splitting**	MEDIUM	932 KB in one chunk for a mobile-first payments app
14	**Test suite drift**	MEDIUM	4 stale failures; ~180 source-shape assertions; server suite needs a live cluster
15	**Branch sprawl**	LOW	17 merged-but-kept local branches

19 Critical Open Issues

P0 Must fix before real money

GA-01 The one-time password is a constant

P0 AUTHENTICATION

EVIDENCE

`server.js:940` — `const prototypeOtp = process.env.PROTOTYPE_OTP || '123456'`. No SMS provider in `server/package.json`. The comment at `:953` confirms: "There is still no SMS gateway here"

ROOT CAUSE

Deliberate prototype decision, never replaced

IMPACT

Full account takeover from `(symbolId, mobileNumber)`: `POST /api/otp/send` purpose `pin_reset` → `POST /api/otp/verify` with `123456` → `POST /api/pin/reset` → a valid 7-day bearer token. Every route the token reaches is then open, including `POST /api/transactions/send`

FILES

`server/server.js:885-1010`, `:1533-1610`

RECOMMENDED FIX

Real SMS delivery with a per-request random code. Keep `PROTOTYPE_OTP` for local and test runs only, and refuse to honour it when `IS_PRODUCTION_DEPLOY` is true — the pattern `AUTH_TOKEN_SECRET` already uses

DEPENDENCIES

An SMS provider account and a delivery budget

TESTING

Delivery; expiry; single-use; the production refusal path; rate limiting under the real flow

GA-02 PIN reset does not revoke other sessions

P0 SESSION MGMT

EVIDENCE

`credentialsInvalidatedAt` is written at exactly one place — `server.js:1453`, inside `POST /api/pin/change`. `POST /api/pin/reset` (`:1533-1610`) writes it nowhere

ROOT CAUSE

Revocation added with PIN *change* and not carried to PIN *reset*

IMPACT

Resetting a PIN — the action taken precisely when a credential may be compromised — leaves every other session valid for up to seven days. With GA-01, an attacker takes the account over and the real owner's devices stay signed in and silent

FILES

`server/server.js:1533-1610`

RECOMMENDED FIX

Stamp `credentialsInvalidatedAt` before minting the reply token, exactly as `pin/change` does

DEPENDENCIES

None. Roughly a two-line change

TESTING

Old token rejected after reset; the reset caller's own new token still works

P1 Major reliability, security or data issue

GA-03 Missing `await` breaks coin local-currency conversion

P1 LIVE NOW

EVIDENCE

`lib/countryCurrency.js:96` is `async`; called without `await` at `server.js:6588` and `:6680`. Live: `"localCurrency":{}`, `"localHeld":null` on every country row

ROOT CAUSE

The helper became `async` when it started reading the seeded Country table; two call sites were not updated

IMPACT

The entire local-currency conversion feature is dead on both holder routes and has never worked. It fails into its own catch, so nothing surfaces

RECOMMENDED FIX

Add `await` at both sites — `:6588` is already inside an async map callback, `:6680` inside an async handler

TESTING

Assert `localCurrency` is a string and `localHeld` a number for a seeded country, both routes

GA-04 Asset-seed interest summed across currencies

P1 · GLB-05 · PRE-EXISTING

EVIDENCE

`server.js:4336` — `totalClaimed += interestAvailable` over seeds whose `currency` differs; credited at `:4356` in the account's own currency. Acknowledged at `:4352` and in `coverageAggregation.js:367`

IMPACT

Creates or destroys value. A USD seed's interest credited as INR at 1:1 understates by ~85×; the reverse overstates by the same factor

RECOMMENDED FIX

Group by `seed.currency`, convert each group through `getRate` into the account's own currency, then sum

TESTING

Mixed-currency seeds; single-currency unchanged; refusal when a rate is unavailable

GA-05 Interest claim is not atomic

P1 · DURABILITY

EVIDENCE

`server.js:4317-4360` — per-seed conditional `findOneAndUpdate` in a loop, then one `User.$inc`, with no session

IMPACT

A crash between the seed writes and the balance credit marks interest paid without paying it. Unrecoverable without manual repair

RECOMMENDED FIX

Wrap in `withMongoTransaction`, passing the session to every write

DEPENDENCIES

GA-04 should land first — same function

GA-06 Share leg and receipts are outside the transaction and not idempotent

P1 · RELIABILITY

EVIDENCE

`server.js:5870-5895` calls `mintShareLegAndReceipts` after the commit. No uniqueness on `metadata.paymentTransactionId`; `Receipt(transactionId, role)` is not unique

IMPACT

A crash after commit leaves a successful payment with no share leg and no receipts, permanently and silently. A re-run would create duplicates with nothing to stop it

RECOMMENDED FIX

Partial unique index on `(type:'share', metadata.paymentTransactionId)` and unique on `Receipt(transactionId, leg, role)`. Then move the step into the transaction, or add a reconciliation job that finds payments with no receipt trail

GA-07 Share-leg receipts pair the wrong currency with the amount

P1 LATENT

EVIDENCE

`merchantShareFlow.js:250-256` — `amount: cashback` (payer's currency) with `currency` (destination). The share Transaction at line 216 correctly uses `cashbackCurrency || currency`

IMPACT

Every cross-border share receipt records a figure under the wrong currency symbol. Latent only because nothing reads the collection — it becomes a live money-display bug the moment anything does

RECOMMENDED FIX

Give `issueReceiptPair` per-role amount and currency

GA-08 No uniqueness constraint on ledger entries or receipts

P1 DATA INTEGRITY

EVIDENCE

`models/LedgerEntry.js:68-69` and `models/Receipt.js:82-83` — compound indexes, neither `unique`

IMPACT

Duplicate ledger rows for one leg of one transaction are structurally possible. The ledger is the audit record; it must be the one thing that cannot double

RECOMMENDED FIX

Unique on `LedgerEntry(transactionId, userId, entryType)` and `Receipt(transactionId, leg, role)`. Scan for existing duplicates first — the scan is itself a finding

GA-09 FX staleness is unbounded

P1 FINANCIAL

EVIDENCE

`lib/fxRates.js:106-128` returns `stale:true` with an arbitrarily old cached rate. `server.js:5175` destructures only `{rate, source}`

IMPACT

If the provider stays down, payments keep settling at a rate that gets older without limit. Auditable via `fxRateSource`, but not gated

RECOMMENDED FIX

Return and read `stale` / `fetchedAt`; refuse a payment whose rate is older than a stated ceiling, as an unresolvable rate is already refused

GA-10 Login and PIN reset write no audit record

P1 OBSERVABILITY

EVIDENCE

15 `recordAudit` call sites. Logged: `transaction.send.*`, `pin.change*`, `security.settings.update`, `project.*`, `geu.*`. **Not logged:** login, registration, PIN set, PIN reset, passkey, coin, profile change, symbolId change, face enroll/verify

IMPACT

The two account-takeover paths (GA-01, GA-02) leave no trace. A compromise cannot be detected or reconstructed

RECOMMENDED FIX

`recordAudit` on every credential and identity route, success and failure. The recorder is already fire-and-forget and cannot fail its caller

GA-11 QR payloads are unsigned, unbound and replayable

P1 QR / PAYMENTS

EVIDENCE

`backend/utils/gloobalQR.js` — 12+7+1 plaintext symbols; the checksum algorithm ships in the bundle. `App.jsx:579` — `usedQrCodes` is an in-memory Set, checked at `:688`

IMPACT

A modified QR redirects funds to any valid Gloobal ID. A request QR can be replayed after the 15-second server window. Replay protection is lost on page reload

RECOMMENDED FIX

Server-minted signed payment requests: an HMAC over `(payeeId, amount, currency, nonce, exp)` issued by a new route and verified server-side at payment time. Keep the identity QR unchanged — it addresses, it does not authorise

P2 Important functional issue

ID	Title	Evidence	Impact
GA-12	Coin holders use the raw country field	`server.js:6563` groups on `$countryIso`, bypassing `accountCountryIso`	Legacy accounts attributed to India; two country rules in one server
GA-13	`totalSent` / `totalReceived` mix currencies	`server.js:6157-6193`	Latent — `App.jsx:1872` reads only `transactions`
GA-14	History has no pagination	`.limit(50)` and `.limit(100)`, no cursor	Older payments unreachable past the cap
GA-15	Coin fiat legs mislabelled (GLB-04)	Hardcoded `reserveCurrency`	A non-INR account's coin ledger lines carry the wrong currency
GA-16	No global error/404 handler, no health route	`GET /` → HTML 404; malformed JSON → HTML. Confirmed live	Response contract broken on the error path; nothing to monitor uptime against
GA-17	`/api/otp/send` is an enumeration oracle	409 / 404 / 400 distinguish three states	Confirms whether a number is registered, and under which country code
GA-18	Rate limiter volatile and per-instance	`server.js:760-790`, self-documented	A free-tier cold start resets every counter
GA-19	`Receipt` is write-only	No route queries it	Storage grows; the receipt trail is unreachable

GA-20	PayLater: no repayment, mixed currencies	`server.js:4434`, `:4393`	Dues only ever grow; the limit is a cross-currency sum
GA-21	Upload bytes never validated	`server.js:4776-4800`	Stored file may not be the recorded type. Mitigated by attachment + nosniff + CSP
GA-22	Financial metadata is unvalidated `Mixed`	`models/Transaction.js`	No casting or type checking on `sourceAmount`, `fxRate`, `cashback`

P3 Quality and maintenance

ID	Title	Evidence
GA-23	Four stale test failures	2 contradict `4fbc7dd`, 2 predate the GC→GEU rename
GA-24	~180 source-shape assertions	Regex over source text rather than behaviour
GA-25	Duplicate `replacedBy` key	`server.js:395` and `:398`
GA-26	932 KB single JS chunk	Vite warns each build
GA-27	Monolithic files	`server.js` 8,274 · `Dashboard.jsx` 4,162 · `App.jsx` 4,000
GA-28	No migration framework	Five ad-hoc scripts
GA-29	Documented known-failures list is stale	`CLAUDE.md` names only `Notification`; the scan also reports `BarcodeDetector` (correctly guarded)
GA-30	Bearer token in `localStorage`	XSS-reachable; mitigated by the frontend CSP
GA-31	17 stale local branches	`git branch -v`
GA-32	Duplicate country data files	`countries.js` and `countries-1.js`, both 321 lines

---

20 Recommended Roadmap

Ordered by the stated priority: financial correctness → data integrity → security → auth → reliability → core function → QR → projects → polish → debt.

NOW — before any real money

#	Task	Issue	Why here
1	Fix the two missing `await`s	GA-03	A confirmed live defect, a two-character change, zero risk. First because it costs nothing
2	Revoke sessions on PIN reset	GA-02	Two lines. Closes half of the takeover path immediately, before the SMS work starts
3	Group asset-seed interest by currency	GA-04	Financial correctness is priority one, and this is the only place still creating or destroying value
4	Wrap the interest claim in a transaction	GA-05	Same function — do both in one change
5	Add uniqueness to LedgerEntry and Receipt	GA-08	Data integrity. Scan for existing duplicates first; the scan is itself a finding
6	**Real SMS OTP delivery**	GA-01	**The production blocker.** Later than 1–5 only because it needs a vendor and a budget, and those five need neither
7	Audit-log every credential route	GA-10	Must land with 6. An authentication change you cannot observe is not a change you can trust
8	Bound FX staleness	GA-09	Financial correctness, needs a policy decision, so it follows the mechanical fixes

NEXT — before wider use

#	Task	Issue	Why here
9	Idempotency and reconciliation for the share/receipt tail	GA-06	Reliability. Needs the indexes from step 5
10	Fix share-receipt currency	GA-07	Correctness, but latent — nothing reads Receipt yet
11	Health route, 404 handler, global error handler	GA-16	Reliability, and a prerequisite for any uptime monitoring
12	Move coin holders onto `accountCountryIso`	GA-12	Core function. One country rule, not two
13	Paginate the history routes	GA-14	Core function. The 50-row cap is already reachable
14	Fix coin ledger currency labels	GA-15	Closes GLB-04
15	Magic-byte validation on uploads	GA-21	Projects hardening
16	Retire the four stale tests	GA-23	A red CI trains people to ignore CI. Cheap, and it unblocks everything after
17	**Signed, server-verified payment-request QR**	GA-11	Deliberately after the above: a new subsystem, where everything before it is a repair to something that already exists

LATER — hardening and debt

#	Task	Issue
18	Shared-store rate limiting (Redis or Mongo-backed)	GA-18
19	Split `server.js` into routers, controllers and services	GA-27
20	Split `Dashboard.jsx` and `App.jsx`; introduce a state layer	GA-27
21	Code-split the bundle	GA-26
22	Migration framework and a documented backup/restore runbook	GA-28 · category O
23	Metrics, tracing and alerting	category H
24	Move attachments to object storage	GA-19 · category J
25	Read `Receipt` — or delete it	GA-19
26	PayLater repayment flow	GA-20
27	Replace source-shape assertions with behavioural ones	GA-24
28	Housekeeping: duplicate key, stale branches, duplicate country files, CLAUDE.md	GA-25, 29, 31, 32

---

21 Exact Files and Modules of Interest

Highest risk

File	Lines	Why
`server/server.js`	8,274	The monolith. Contains GA-01, GA-02, GA-03, GA-04, GA-05, GA-12…GA-17
`server/lib/merchantShareFlow.js`	267	GA-06, GA-07
`server/lib/fxRates.js`	~135	GA-09
`backend/utils/gloobalQR.js`	~190	GA-11
`frontend/App.jsx`	4,000	GA-11 (client half), GA-27
`server/models/LedgerEntry.js`	~70	GA-08
`server/models/Receipt.js`	~85	GA-08, GA-19
`server/models/Transaction.js`	~140	GA-22

Highest quality — read these to understand the system

File	Lines	Why
`server/lib/coverageAggregation.js`	551	The best module here. Refuses to fabricate Our Spending and says why
`server/lib/settlementEngine.js`	526	Conditional `$inc`, ordered writes, self-describing revert
`server/lib/accountCountry.js`	–	One country rule, shared by every consumer that matters
`server/lib/countryCurrency.js`	116	Seeded table with a bundled fallback and no invented currency
`server/lib/auditTrail.js`	–	Fire-and-forget that cannot fail its caller
`backend/services/api/httpClient.js`	–	401 vs `status===0`. Why a cold start does not sign people out
`server/server.js:5540-5660`	120	`performTransfer`. The core of the product

Key line references

Location	Subject
`server/server.js:940`	GA-01 — the constant OTP
`server/server.js:1453`	The only `credentialsInvalidatedAt` write
`server/server.js:1533`	GA-02 — PIN reset, no revocation
`server/server.js:4336`	GA-04 — cross-currency interest sum
`server/server.js:5175`	GA-09 — `stale` discarded
`server/server.js:6588`, `:6680`	GA-03 — the missing `await`s
`server/server.js:6563`	GA-12 — raw country field
`server/lib/merchantShareFlow.js:250`	GA-07 — wrong currency on share receipts
`frontend/App.jsx:579`, `:688`	GA-11 — client-side replay guard
`server/models/Transaction.js:130-138`	The idempotency index — a strength

---

22 Evidence and Verification

What was executed

Check	Result
`git status` / `log` / `branch`	Clean, in sync, 105 commits
`node build_app.mjs`	Built from 74 consolidated imports
`npm run build` (Vite production)	`dist/assets/index-B8AepYrZ.js`
`node --test "tests/*.test.mjs"`	**695 tests · 693 pass · 2 fail** (420s)
`node --test financial-principles-tests/tests/*`	**215 tests · 213 pass · 2 fail** (70s)
`tools/frontend/scan-undeclared.mjs`	3,236 bindings; 2 undeclared, both guarded
`tools/frontend/probe-screens.mjs`	Coverage 388/388 · AddBank 194/194 · stored-selection 194/194 · 2 expected SSR failures
`tools/frontend/probe-panels.mjs`	3/3

`tools/frontend/probe-stages.mjs`	Pre-existing regex failure, as documented
`tools/backend/check-backend.mjs`	Backend reachable and answering correctly

Production reads — all `GET`, no writes

Endpoint	Result
`GET /api/coverage`	200 in 23.2s (cold start). 120 users, 20 countries, 188 payments, 85 exact rows
`GET /api/stats`	200. `totalUsers: 120` — matches coverage exactly
`GET /api/coin/holders`	200. `issued === heldByAccounts === 121373.61`. `localCurrency:{}` **on every row**
`GET /api/profile/TESTID`	401
`GET /api/transactions/history/TESTID`	401
`GET /api/projects`	200 — published only
`GET /api/geu/supply`	503 — correctly disabled
`GET /api/users/available?symbolId=x`	400 — malformed ID refused
`GET /`	404, HTML
`POST /api/login` with malformed JSON	400, HTML
Response headers, both targets	All expected headers present
`GET https://gloobalv3.netlify.app/`	Serves `/assets/index-B8AepYrZ.js` — **matches the local build of HEAD**

No account was created. No payment was made. No write of any kind was issued.

What was not verified, and why

Not verified	Reason
`server/tests/` — 21 files	Each creates and drops a database on the production Atlas cluster. Excluded by the read-only terms
The exact/reconstructable/approximate split within the 103 rows	Needs a database read. No route exposes it
Whether any historical row has conflicting amounts or currencies	Same
Whether `backfill-country-iso.mjs` has been run in production	Cannot be determined from outside
GLB-12…GLB-25 test evidence	Lives in the unrun server suite. The *fixes* were verified by reading the code
`/api/coin/holders/:countryIso` GA-03 behaviour	Requires an authenticated token. The code is identical to the confirmed site

> **Standard applied**
>
> Every conclusion rests on source code, a schema, executed test output, a production read, or git history. Where evidence was unavailable, the finding is marked **NOT VERIFIED** rather than inferred. No PII appears anywhere in this report.

---

23 Final Risk Summary

Risk	Likelihood	Impact	Overall
Account takeover via the constant OTP	**Certain** if targeted	Critical	🔴 **CRITICAL**
Compromise undetectable (no audit on auth routes)	**Certain**	High	🔴 **CRITICAL**
Session survives a PIN reset	**Certain**	High	🔴 **CRITICAL**
Value created/destroyed by cross-currency interest sums	Medium	High	🟠 **HIGH**
Receipt trail permanently lost after a crash	Low	High	🟠 **HIGH**
Duplicate ledger rows	Low	High	🟠 **HIGH**
Funds redirected by a tampered QR	Medium	High	🟠 **HIGH**
Data loss with no restore runbook	Low	Critical	🟠 **HIGH**
Payment settled at an indefinitely stale rate	Low	Medium	🟡 **MEDIUM**
Coin local-currency display dead	**Certain — live now**	Low	🟡 **MEDIUM**
History unreachable past the row cap	Medium	Medium	🟡 **MEDIUM**
Cold-start latency read as failure	High	Low	🟢 **LOW**
Concurrent balance corruption	Very low	Critical	🟢 **LOW — properly defended**
Duplicate payment from a retry	Very low	High	🟢 **LOW — database-enforced**

The judgement

The money is safe. The account is not.

The transfer path is genuinely well engineered — atomic, idempotent, currency-correct, and defended at the database level rather than in application logic. Nothing found in this audit threatens the integrity of a payment once it starts.

Getting to the point of starting one is the problem. The second factor is a constant, the reset path leaves old sessions alive, and neither writes an audit record. Those three are one story, and they are the entire distance between this system and a production payments platform.

Eight tasks in the NOW column close it. Five of them are small. One needs a vendor.

Audit performed 9 September 2026 against commit `0d3989e`.
Read-only throughout: no application source was modified, no temporary file remains, and nothing was committed, pushed or deployed.